

JAVA AWT CONTAINER COMPOSITION AND LAYOUT

Course code: CSA-0924

Course: Java Programming

MANAGERS

Program Demonstration – Two Questions

Name: Sakthivel S

Reg.no:192524378

QUESTION 1

Question: A desktop application's main window nests a Panel containing a toolbar, using FlowLayout, inside a larger Frame that uses BorderLayout for its overall structure. Explain the general AWT container-composition principle that makes it valid to combine two different layout managers within one window this way.

AIM

To demonstrate the AWT container-composition principle by placing a Panel with FlowLayout inside a Frame controlled by BorderLayout.

ALGORITHM

1. Create a Frame to act as the main application window.
2. Set the Frame's layout manager to BorderLayout.
3. Create a Panel to represent the toolbar container.
4. Set the Panel's layout manager to FlowLayout.
5. Add toolbar components such as buttons to the Panel.
6. Add the Panel to the NORTH region of the Frame using BorderLayout.
7. Set the Frame size and make it visible.
8. Observe that each container manages the arrangement of only its own child components.

PSEUDO CODE

```
START
    Create Frame
    Set Frame layout to BorderLayout

    Create Panel
    Set Panel layout to FlowLayout

    Create toolbar buttons
    Add buttons to Panel

    Add Panel to Frame at NORTH
    Set Frame size
    Display Frame
STOP
```

JAVA CODE

```
import java.awt.*;
import java.awt.event.*;

public class AWTContainerComposition1 {
    public static void main(String[] args) {
        Frame frame = new Frame("AWT Container Composition");

        // Parent container uses BorderLayout
        frame.setLayout(new BorderLayout());

        // Child container uses FlowLayout
        Panel toolbar = new Panel();
        toolbar.setLayout(new FlowLayout(FlowLayout.LEFT));

        toolbar.add(new Button("New"));
        toolbar.add(new Button("Open"));
        toolbar.add(new Button("Save"));
        toolbar.add(new Button("Exit"));

        // Add the child container to the parent's NORTH region
        frame.add(toolbar, BorderLayout.NORTH);

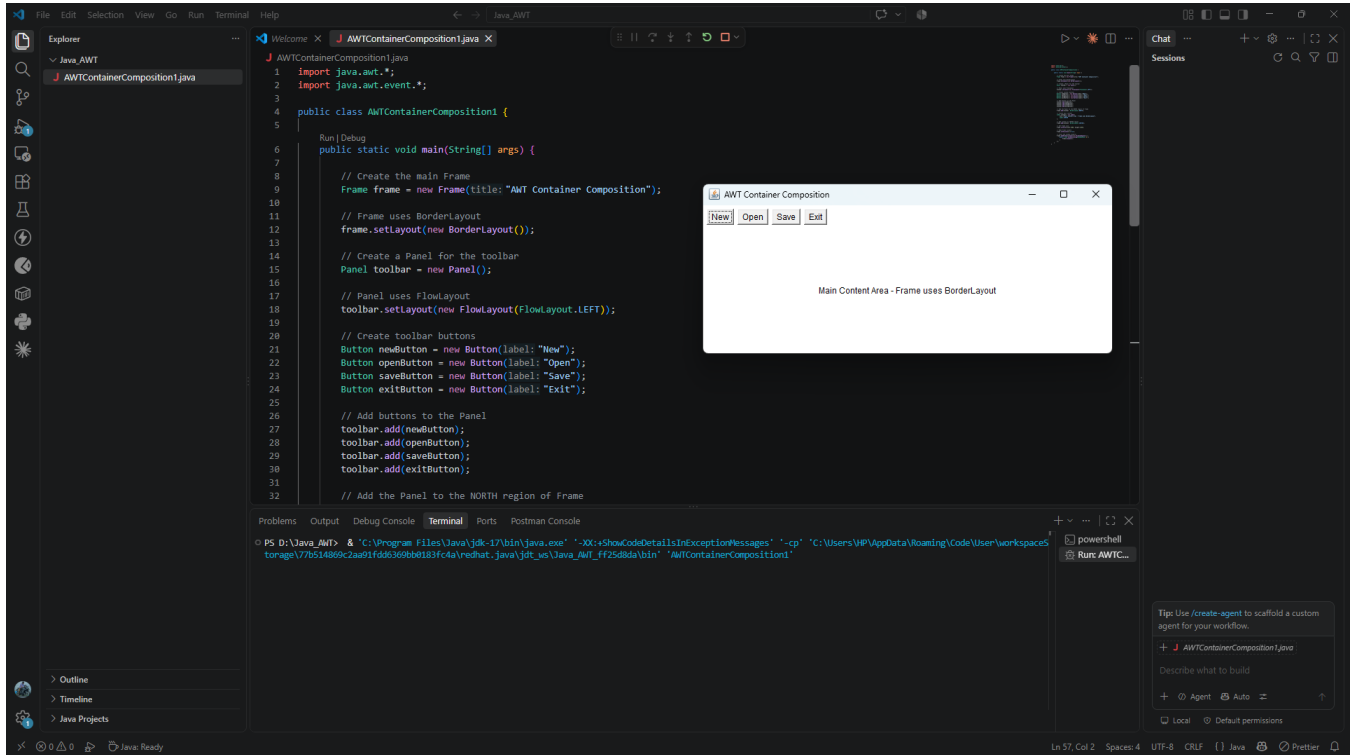
        Label content = new Label(
            "Main content area - Frame uses BorderLayout",
            Label.CENTER
        );
        frame.add(content, BorderLayout.CENTER);

        frame.setSize(600, 250);
        frame.setVisible(true);

        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                frame.dispose();
            }
        });
    }
}
```

OUTPUT

A window is displayed with a toolbar at the top. The toolbar buttons are arranged horizontally by `FlowLayout`, while the main Frame positions the toolbar and content using `BorderLayout`.



RESULT

Thus, the AWT container-composition principle is verified: each Container can have its own layout manager, so a Panel using `FlowLayout` can be nested inside a Frame using `BorderLayout`. The parent manages the Panel as one component, while the Panel independently manages its child controls.

QUESTION 2

Question: A desktop reporting tool nests a Panel using FlowLayout, holding filter controls, inside a larger Frame governed by BorderLayout for the overall window structure. Explain the general AWT principle that makes combining two different layout managers within a single window like this valid and common practice.

AIM

To demonstrate how nested AWT containers can independently use different layout managers by placing filter controls in a FlowLayout Panel inside a BorderLayout Frame.

ALGORITHM

1. Create a Frame for the reporting application's main window.
2. Set the Frame's layout manager to BorderLayout.
3. Create a Panel for report filter controls.
4. Set the Panel's layout manager to FlowLayout.
5. Create filter controls such as a label, text field, and buttons.
6. Add the filter controls to the Panel.
7. Add the Panel to the NORTH region of the Frame.
8. Add a report area to the CENTER region.
9. Set the Frame size and display the window.
10. Verify that the two containers independently arrange their own components.

PSEUDO CODE

```
START
    Create Frame
    Set Frame layout to BorderLayout

    Create Filter Panel
    Set Filter Panel layout to FlowLayout

    Create filter controls
    Add filter controls to Filter Panel

    Add Filter Panel to Frame at NORTH
    Create report display area
    Add report area to Frame at CENTER

    Set Frame size
    Display Frame
STOP
```

JAVA CODE

```
import java.awt.*;
import java.awt.event.*;

public class AWTContainerComposition2 {
    public static void main(String[] args) {
        Frame frame = new Frame("Reporting Tool - AWT Layouts");

        // Main window uses BorderLayout
        frame.setLayout(new BorderLayout(10, 10));

        // Filter container uses FlowLayout
        Panel filterPanel = new Panel(
            new FlowLayout(FlowLayout.LEFT, 8, 8)
        );

        filterPanel.add(new Label("Report:"));
        filterPanel.add(new TextField(15));
        filterPanel.add(new Button("Apply"));
        filterPanel.add(new Button("Clear"));

        // BorderLayout positions the entire filter panel
        frame.add(filterPanel, BorderLayout.NORTH);

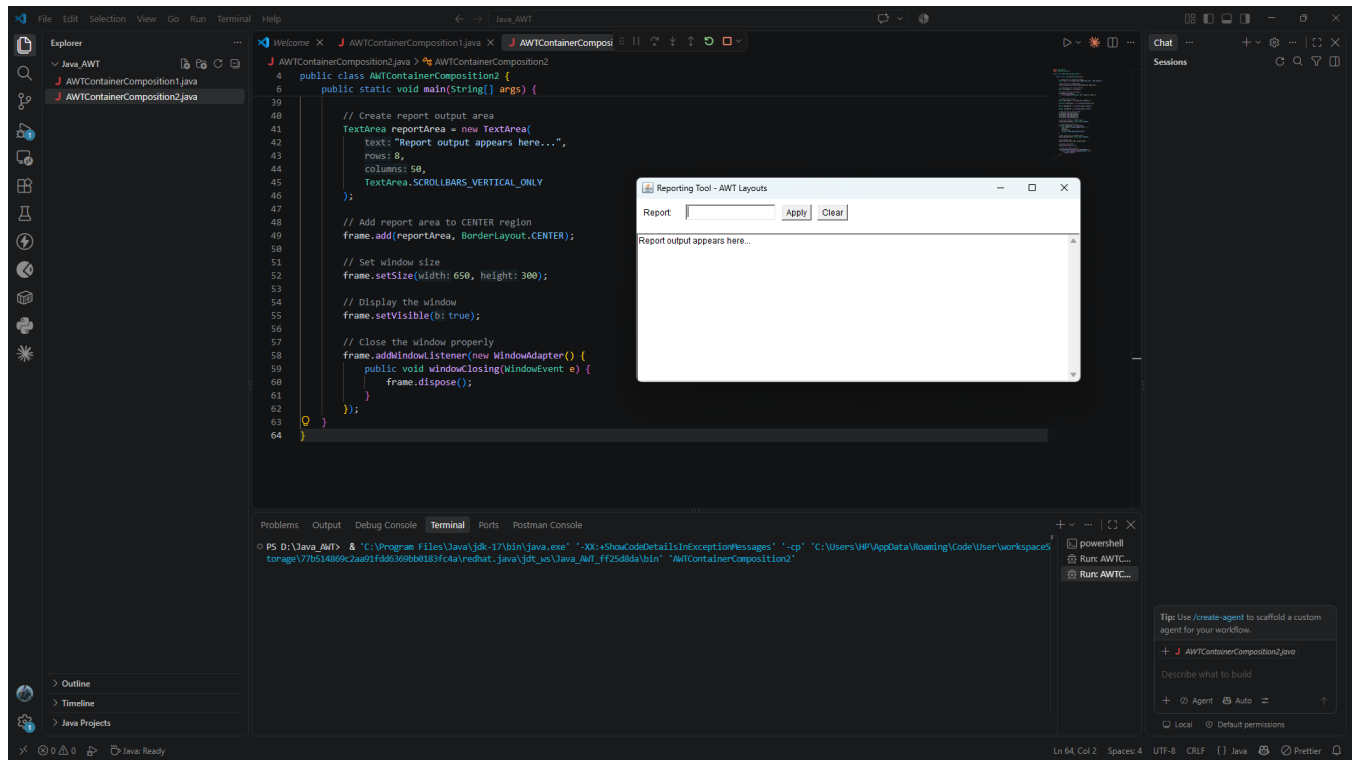
        TextArea reportArea = new TextArea(
            "Report output appears here...",
            8, 50,
            TextArea.SCROLLBARS_VERTICAL_ONLY
        );
        frame.add(reportArea, BorderLayout.CENTER);

        frame.setSize(650, 300);
        frame.setVisible(true);

        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                frame.dispose();
            }
        });
    }
}
```

OUTPUT

A reporting window is displayed with the filter controls arranged in a row at the top. FlowLayout controls the arrangement inside the filter Panel, while BorderLayout places the complete Panel at the top and the report area in the center.



RESULT

Thus, the AWT principle is verified: layout managers belong to individual containers, not to the entire window. Nested containers may therefore use different layout managers, allowing complex interfaces to be constructed by combining simple independently managed sections.